

Expressions and operator order

Project: Python readiness portfolio

1. Your mission

Your learning target

I can evaluate numeric expressions using Python's grouping rules.

I can use parentheses to express the intended calculation.

I can distinguish ordinary division, whole groups and a remainder.

Remember before reading

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Values variables and types. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the evidence shows was wrong.

This week you will practise:

Evaluate numeric expressions using Python's grouping rules

Use parentheses to express the intended calculation

Distinguish ordinary division, whole groups and a remainder

Retrieve what you already know

What is the difference between $8 + 2$ and $"8" + "2"$?

My starting explanation

This week's end goal

Trace expressions and operator order and explain the changed program before running it.

2. Notice the evidence

Three packs each contain eight counters, with two more loose counters. First count the packed counters, then add the loose ones. A program needs that structure, not just the right numbers. This week you will predict expressions, make grouping visible and check division against a physical interpretation.

Use numeric values in a fresh Python 3 script called expressions.py. Keep expected and observed results separate as before. Multiplication and division group more tightly than addition and subtraction. Parentheses let you state a different grouping. A correct calculation still depends on correct quantities and units.

List the given facts and one fact that is not supplied.

3. See how it works

Two expressions with different meanings

Expression · First calculation · Final value

Expression: $3 * 8 + 2$

First calculation: $3 * 8$ is 24

Final value: $24 + 2$ is 26

Expression: $3 * (8 + 2)$

First calculation: $8 + 2$ is 10

Final value: $3 * 10$ is 30

Expression: $(3 * 8) + 2$

First calculation: $3 * 8$ is 24

Final value: $24 + 2$ is 26

Adding two inside every pack differs from adding two loose counters once.

```
packs = 3
per_pack = 8
loose = 2
total = packs * per_pack + loose
print(total)
```

The script displays 26. Replacing the total expression with $\text{packs} * (\text{per_pack} + \text{loose})$ displays 30, because it adds two to every pack. Parentheses are not decoration: they can change the represented situation. Write a brief phrase for each group before running.

Connect the picture and the explanation

Point to the input, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

4. Compare the cases

Compare a familiar case with a changed case

Predict $16 - 5 - 3$, $16 - (5 - 3)$, and $18 / 3 * 2$. Write the grouping, run each line and record actual results separately.

Keep the procedure fixed. Explain what changed in the inputs and result.

Change one thing in the pictured case

Change: Replace the final multiplier 5 with 4 in both expressions.

Prediction: The first result is 11; the grouped result is 20.

Reason: $3+2*4=3+8$; $(3+2)*4=5*4$.

5. Words that unlock the idea

Word · Meaning

Word: Expression

Meaning: Code evaluated to produce a value

Word: Operator

Meaning: A symbol such as + or * that specifies an operation

Word: Precedence

Meaning: Rules that determine how an expression is grouped

Word: Parentheses

Meaning: Symbols that make a grouping explicit

Word: Remainder

Meaning: The amount left after forming whole groups

Use two terms to explain a step in this week's example.

6. Follow a worked example

```
print(20 - 6 - 2)

print(20 - (6 - 2))

print(12 / 3 * 2)
```

The first line groups subtraction from left to right: $(20 - 6) - 2$ gives 12. The second gives 16. Multiplication and division share a precedence level, so $12 / 3 * 2$ groups as $(12 / 3) * 2$ and displays 8.0, not 2.0. Ordinary / division produces a float, even when the result is mathematically whole.

Explain why $18 - 4 * 3$ gives 6 while $(18 - 4) * 3$ gives 42. Which operation is grouped first in each expression?

Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

A pictorial worked case: Expressions and operator order

Parentheses change an expression. A toy shelf calculator compares adding a fixed cost after multiplication with multiplying an entire group.

The facts we will use

Python 3: `print(3+2*5); print((3+2)*5)`. Inputs are dimensionless for this arithmetic exercise.

Expressions and operator order

Start with the supplied facts

Follow each arrow. Ask which fact allows the next step.



1. Use precedence

In $3+2*5$, multiplication happens first.



2. Compute first result

$2*5$ is 10; adding 3 gives 13.



3. Compute grouped result

Parentheses give $3+2=5$ first; multiplying by 5 gives 25.

CHANGE ONE THING

Replace the final multiplier 5 with 4 in both expressions.

Explain the changed result using the matching lesson.

Read the picture one step at a time

Step · What happens and why

Step: 1. Use precedence

What happens and why: In $3+2*5$, multiplication happens first.

Step: 2. Compute first result

What happens and why: $2*5$ is 10; adding 3 gives 13.

Step: 3. Compute grouped result

What happens and why: Parentheses give $3+2=5$ first; multiplying by 5 gives 25.

Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

Why this result follows

Parentheses explicitly group an operation. Without them, Python multiplication takes priority over addition; the same numerals can therefore describe different procedures.

What this example does not establish

Arithmetic correctness does not decide which expression matches a real problem; the written model must define the intended quantities.

7. Find and repair a mistake

```
counters = 17
group_size = 5
print(counters / group_size)
print(counters // group_size)
print(counters % group_size)
```

The outputs are 3.4, 3 and 2. With nonnegative whole counters and a positive group size, `//` gives the number of complete groups and `%` gives the remainder: $17 = 3 * 5 + 2$. Ordinary division answers a different question about a ratio; 3.4 complete five-counter groups cannot be built without splitting counters.

For nonnegative whole inputs, the remainder is at least zero and smaller than the group size. A group size of zero is invalid and division by zero raises an error. These whole-group interpretations assume nonnegative counts and positive group sizes; do not generalise them to negative quantities without examining Python's floor-division rule.

Locate the earliest unsupported step

For 17 counters in groups of 5, explain `/`, `//` and `%`. Show a multiplication-and-addition check for the complete groups and remainder.

State one repair and the checks you would rerun.

8. Practise together

Write an expression for three packs of eight counters plus two loose counters. Explain why adding two within the parentheses for every pack describes a different situation.

Explain why $18 - 4 * 3$ gives 6 while $(18 - 4) * 3$ gives 42. Which operation is grouped first in each expression?

Explain your prediction to a partner before comparing results.

Practise with less help: Predict then test expressions

Use workbook W2 for the connected example “Expressions and operator order”. First fill the input and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

Plan · Do · Check

Plan: What is given? Which rule or procedure applies?

Do: Record each intermediate step.

Check: Does the result follow? What remains unknown?

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case.

Each person writes their own explanation.

Check your reasoning

Use the worked case for Expressions and operator order. Proposed explanation: “All operations are read left to right regardless of precedence”. Decide whether this explanation is supported. Point to the step or supplied fact that decides; explain your repair if needed.

Answer on your own before discussion. If you are unsure, say which step you need help with.

Explain the picture

Trace the operation order in both expressions. Identify which picture step changes with parentheses and predict both results when the multiplier becomes four.

My explanation using a pictured step and the given facts

This is supported practice. Keep the separately named independent case for an attempt before feedback.

9. Run the investigation

Predict, investigate and preserve the evidence

Predict $16 - 5 - 3$, $16 - (5 - 3)$, and $18 / 3 * 2$. Write the grouping, run each line and record actual results separately.

```
packs = 3
per_pack = 8
loose = 2
total = packs * per_pack + loose
print(total)
```

```
print(20 - 6 - 2)
print(20 - (6 - 2))
print(12 / 3 * 2)
```

```
counters = 17
```

```
group_size = 5

print(counters / group_size)

print(counters // group_size)

print(counters % group_size)
```

Record · Your evidence

Record: Given input and fixed rule

Your evidence: _____

Record: Prediction before checking

Your evidence: _____

Record: Actual result, or “not run”

Your evidence: _____

Record: First mismatch and one explanation

Your evidence: _____

Record: Boundary or missing input and expected response

Your evidence: _____

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Plan → try → check

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

10. Build your understanding

Explain → apply → challenge

Write an expression for three packs of eight counters plus two loose counters. Explain why adding two within the parentheses for every pack describes a different situation.

For 17 counters in groups of 5, explain /, // and %. Show a multiplication-and-addition check for the complete groups and remainder.

Explain why a group size of zero must be excluded. State the assumptions that let complete groups and a remainder describe the packing task.

Explain which evidence supports your conclusion and which claim remains unproved.

11. Connect your project

Add a labelled expressions and operator order evidence sheet to Python readiness portfolio. Show how you can evaluate numeric expressions using python’s grouping rules. Use the supplied fictional case and keep its inputs, intermediate steps, calculation or prediction, and limitation. Connect this record to last week’s record; a matching answer without a trace is insufficient.

Project evidence · Record

Project evidence: Component and version

Record: _____

Project evidence: Inputs permitted at prediction time

Record: _____

Project evidence: Trace and comparison with earlier work

Record: _____

Project evidence: Failure, limit and human review

Record: _____

Evidence for your project

For Python readiness before the Responsible Plant Classifier, retain a trace of the values, the predicted outputs and the changed-case explanation. Use actual Python 3 results to check the prediction after reasoning first.

12. Show what you know

Keep the expression, substituted numbers, grouping, units and actual result. A bare number does not show why it answers the question. If an edit changes the grouping, preserve the old result and rerun a case where the expressions differ. Week 4 will turn comparisons into Boolean conditions.

A team has 23 whole tiles and puts 4 in each complete tray. Which operations find complete trays and leftover tiles? Predict both values and verify them by reconstructing 23. Explain why / alone does not answer both questions.

Explain why a group size of zero must be excluded. State the assumptions that let complete groups and a remainder describe the packing task.

My limited claim and a useful next check

13. Study a complete case

Complete evidence record

Two expressions with different meanings

Expression · First calculation · Final value

Expression: $3 * 8 + 2$

First calculation: $3 * 8$ is 24

Final value: $24 + 2$ is 26

Expression: $3 * (8 + 2)$

First calculation: $8 + 2$ is 10

Final value: $3 * 10$ is 30

Expression: $(3 * 8) + 2$

First calculation: $3 * 8$ is 24

Final value: $24 + 2$ is 26

Adding two inside every pack differs from adding two loose counters once.

Follow the input through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

My complete trace of the worked case

14. Try a new case independently

Independent case: Group an expression deliberately

Predict each line. Explain which calculation groups addition before division. What would $(17+5)\%5$ return, and how do $/$, $//$ and $\%$ differ for the supplied positive integers?

```
print((17 + 5) // 5)
print(17 + 5 // 5)
print(17 / 5, 17 // 5, 17 % 5)
```

My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

Attempt this without hints

Four trays hold six tiles each and five tiles are loose. Write and evaluate the total expression. Then pack the total into complete groups of four and find the remainder. Explain why $4 * (6 + 5)$ is wrong for this story.

My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

Use feedback to improve

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed. A later check uses a changed case, not a copied answer.